

Admin Analytics Dashboard - Complete Implementation Guide

Project Overview

This comprehensive guide will help you build a production-ready Admin Analytics Dashboard for a CV/Resume analysis platform using the **MERN stack with Next.js**. The dashboard displays platform-wide statistics including user demographics, CV analysis metrics, feedback analytics, and user segmentation.

Technical Stack

Core Technologies

- **Frontend Framework:** Next.js 15 with App Router and Server-Side Rendering (SSR)
- **UI Library:** ShadCN UI (Radix UI + Tailwind CSS)
- **Styling:** Tailwind CSS
- **Charts:** Recharts (recommended), Chart.js, or ECharts
- **Backend Runtime:** Node.js with Next.js API Routes
- **Database:** MongoDB with Mongoose ODM
- **Authentication:** JWT with Next.js Middleware
- **Type Safety:** TypeScript
- **Deployment:** Vercel

Phase 1: Project Setup (Day 1, Hours 0-2)

Step 1: Initialize Next.js Project

```
# Create new Next.js project with TypeScript
npx create-next-app@latest admin-dashboard --typescript --tailwind --app --eslint

# Navigate to project directory
cd admin-dashboard
```

Configuration prompts:

- TypeScript: Yes
- ESLint: Yes
- Tailwind CSS: Yes

- `src/` directory: Yes
- App Router: Yes
- Import alias: `@/*` (default)

Step 2: Install Dependencies

```
# ShadCN UI and components
npx shadcn-ui@latest init

# Install additional packages
npm install mongoose bcryptjs jsonwebtoken
npm install recharts
npm install @types/bcryptjs @types/jsonwebtoken --save-dev

# Install ShadCN UI components
npx shadcn-ui@latest add button card table input label
npx shadcn-ui@latest add dropdown-menu avatar sidebar
```

Step 3: Configure Environment Variables

Create `.env.local` file in root directory:

```
# MongoDB Connection
MONGODB_URI=mongodb+srv://username:password@cluster.mongodb.net/cv-analytics?retryWrites=

# JWT Secret (generate a strong random string)
JWT_SECRET=your-super-secret-jwt-key-min-32-characters

# Application
NEXT_PUBLIC_APP_URL=http://localhost:3000
NODE_ENV=development
```

Important: Never commit `.env.local` to Git. Add it to `.gitignore`.

Phase 2: MongoDB Schema & Connection (Day 1, Hours 2-4)

Step 1: Create Database Connection Utility

File: `src/lib/db/connection.ts`

```
import mongoose from 'mongoose';

const MONGODB_URI = process.env.MONGODB_URI!;

if (!MONGODB_URI) {
  throw new Error('Please define MONGODB_URI in .env.local');
}
```

```

interface MongooseCache {
  conn: typeof mongoose | null;
  promise: Promise<typeof mongoose> | null;
}

declare global {
  var mongoose: MongooseCache | undefined;
}

let cached: MongooseCache = global.mongoose || { conn: null, promise: null };

if (!global.mongoose) {
  global.mongoose = cached;
}

async function connectDB() {
  if (cached.conn) {
    return cached.conn;
  }

  if (!cached.promise) {
    const opts = {
      bufferCommands: false,
    };

    cached.promise = mongoose.connect(MONGODB_URI, opts);
  }

  try {
    cached.conn = await cached.promise;
  } catch (e) {
    cached.promise = null;
    throw e;
  }

  return cached.conn;
}

export default connectDB;

```

Step 2: Define Mongoose Models

File: src/lib/db/models/User.ts

```

import mongoose, { Schema, models } from 'mongoose';

export interface IUser {
  name: string;
  email: string;
  password: string;
  role: 'admin' | 'user';
  country: string;
  careerStage: 'Fresher' | 'Graduate' | 'Experienced';
  subscriptionType: 'paid' | 'free';
  createdAt: Date;
}

```

```

    updatedAt: Date;
  }

const UserSchema = new Schema<IUser>({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['admin', 'user'], default: 'user' },
  country: { type: String, required: true },
  careerStage: {
    type: String,
    enum: ['Fresher', 'Graduate', 'Experienced'],
    required: true
  },
  subscriptionType: {
    type: String,
    enum: ['paid', 'free'],
    default: 'free'
  },
}, { timestamps: true });

export default models.User || mongoose.model<IUser>('User', UserSchema);

```

File: src/lib/db/models/CVAnalysis.ts

```

import mongoose, { Schema, models } from 'mongoose';

export interface ICVAnalysis {
  userId: mongoose.Types.ObjectId;
  cvScore: number;
  analysisDate: Date;
  skills: string[];
  experience: number;
  education: string;
}

const CVAnalysisSchema = new Schema<ICVAnalysis>({
  userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
  cvScore: { type: Number, required: true, min: 0, max: 100 },
  analysisDate: { type: Date, default: Date.now },
  skills: [{ type: String }],
  experience: { type: Number },
  education: { type: String },
});

export default models.CVAnalysis ||
  mongoose.model<ICVAnalysis>('CVAnalysis', CVAnalysisSchema);

```

File: src/lib/db/models/Feedback.ts

```

import mongoose, { Schema, models } from 'mongoose';

export interface IFeedback {
  userId: mongoose.Types.ObjectId;

```

```

    rating: number;
    category: 'UI/UX' | 'Accuracy' | 'Speed' | 'Support';
    comment?: string;
    createdAt: Date;
  }

  const FeedbackSchema = new Schema<IFeedback>({
    userId: { type: Schema.Types.ObjectId, ref: 'User', required: true },
    rating: { type: Number, required: true, min: 1, max: 5 },
    category: {
      type: String,
      enum: ['UI/UX', 'Accuracy', 'Speed', 'Support'],
      required: true
    },
    comment: { type: String },
    createdAt: { type: Date, default: Date.now },
  });

  export default models.Feedback ||
    mongoose.model<IFeedback>('Feedback', FeedbackSchema);

```

Phase 3: Authentication System (Day 1, Hours 4-8)

Step 1: Create JWT Utilities

File: src/lib/auth/jwt.ts

```

import jwt from 'jsonwebtoken';

const JWT_SECRET = process.env.JWT_SECRET!;

export interface JWTPayload {
  userId: string;
  email: string;
  role: string;
}

export function signToken(payload: JWTPayload): string {
  return jwt.sign(payload, JWT_SECRET, { expiresIn: '7d' });
}

export function verifyToken(token: string): JWTPayload {
  try {
    return jwt.verify(token, JWT_SECRET) as JWTPayload;
  } catch (error) {
    throw new Error('Invalid token');
  }
}

```

File: src/lib/auth/password.ts

```
import bcrypt from 'bcryptjs';

export async function hashPassword(password: string): Promise<string> {
  return await bcrypt.hash(password, 10);
}

export async function comparePassword(
  password: string,
  hash: string
): Promise<boolean> {
  return await bcrypt.compare(password, hash);
}
```

Step 2: Create Authentication API Routes

File: src/app/api/auth/register/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import connectDB from '@lib/db/connection';
import User from '@lib/db/models/User';
import { hashPassword } from '@lib/auth/password';
import { signToken } from '@lib/auth/jwt';

export async function POST(request: NextRequest) {
  try {
    await connectDB();

    const body = await request.json();
    const { name, email, password, country, careerStage } = body;

    // Validate input
    if (!name || !email || !password) {
      return NextResponse.json(
        { error: 'Missing required fields' },
        { status: 400 }
      );
    }

    // Check if user exists
    const existingUser = await User.findOne({ email });
    if (existingUser) {
      return NextResponse.json(
        { error: 'User already exists' },
        { status: 409 }
      );
    }

    // Hash password and create user
    const hashedPassword = await hashPassword(password);
    const user = await User.create({
      name,
      email,
      password: hashedPassword,
      country: country || 'Unknown',
    });
  } catch (error) {
    return NextResponse.json(
      { error: 'Internal server error' },
      { status: 500 }
    );
  }
}
```

```

        careerStage: careerStage || 'Fresher',
        role: 'user',
        subscriptionType: 'free',
    });

    // Generate JWT token
    const token = signToken({
        userId: user._id.toString(),
        email: user.email,
        role: user.role,
    });

    return NextResponse.json({
        success: true,
        token,
        user: {
            id: user._id,
            name: user.name,
            email: user.email,
            role: user.role,
        },
    });
} catch (error) {
    console.error('Registration error:', error);
    return NextResponse.json(
        { error: 'Registration failed' },
        { status: 500 }
    );
}
}

```

File: src/app/api/auth/login/route.ts

```

import { NextRequest, NextResponse } from 'next/server';
import connectDB from '@/lib/db/connection';
import User from '@/lib/db/models/User';
import { comparePassword } from '@/lib/auth/password';
import { signToken } from '@/lib/auth/jwt';

export async function POST(request: NextRequest) {
    try {
        await connectDB();

        const body = await request.json();
        const { email, password } = body;

        // Validate input
        if (!email || !password) {
            return NextResponse.json(
                { error: 'Email and password required' },
                { status: 400 }
            );
        }

        // Find user

```

```

const user = await User.findOne({ email });
if (!user) {
  return NextResponse.json(
    { error: 'Invalid credentials' },
    { status: 401 }
  );
}

// Verify password
const isValid = await comparePassword(password, user.password);
if (!isValid) {
  return NextResponse.json(
    { error: 'Invalid credentials' },
    { status: 401 }
  );
}

// Generate JWT token
const token = signToken({
  userId: user._id.toString(),
  email: user.email,
  role: user.role,
});

return NextResponse.json({
  success: true,
  token,
  user: {
    id: user._id,
    name: user.name,
    email: user.email,
    role: user.role,
  },
});
} catch (error) {
  console.error('Login error:', error);
  return NextResponse.json(
    { error: 'Login failed' },
    { status: 500 }
  );
}
}

```

Step 3: Create Middleware for Route Protection

File: src/middleware.ts

```

import { NextRequest, NextResponse } from 'next/server';
import { verifyToken } from '@lib/auth/jwt';

export function middleware(request: NextRequest) {
  const token = request.cookies.get('token')?.value;
  const { pathname } = request.nextUrl;

  // Public routes

```



```

const publicRoutes = ['/login', '/register', '/'];
if (publicRoutes.includes(pathname)) {
  return NextResponse.next();
}

// Protected routes
if (pathname.startsWith('/dashboard') || pathname.startsWith('/admin')) {
  if (!token) {
    return NextResponse.redirect(new URL('/login', request.url));
  }

  try {
    const payload = verifyToken(token);

    // Admin-only routes
    if (pathname.startsWith('/admin') && payload.role !== 'admin') {
      return NextResponse.redirect(new URL('/dashboard', request.url));
    }

    return NextResponse.next();
  } catch (error) {
    return NextResponse.redirect(new URL('/login', request.url));
  }
}

return NextResponse.next();
}

export const config = {
  matcher: ['/(?!api|_next/static|_next/image|favicon.ico).*'],
};

```

Phase 4: Analytics API Routes (Day 2, Hours 8-12)

User Analytics API

File: src/app/api/analytics/users/route.ts

```

import { NextRequest, NextResponse } from 'next/server';
import connectDB from '@lib/db/connection';
import User from '@lib/db/models/User';

export async function GET(request: NextRequest) {
  try {
    await connectDB();

    // Total users
    const totalUsers = await User.countDocuments();
    const paidUsers = await User.countDocuments({ subscriptionType: 'paid' });
    const freeUsers = await User.countDocuments({ subscriptionType: 'free' });

    // Users by country
    const usersByCountry = await User.aggregate([

```

```

    {
      $group: {
        _id: '$country',
        count: { $sum: 1 },
      },
    },
    {
      $project: {
        country: '$_id',
        count: 1,
        _id: 0,
      },
    },
    { $sort: { count: -1 } }
  ]
]);

// Users by career stage
const usersByCareerStage = await User.aggregate([
  {
    $group: {
      _id: '$careerStage',
      count: { $sum: 1 },
    },
  },
  {
    $project: {
      stage: '$_id',
      count: 1,
      _id: 0,
    },
  },
]);

return NextResponse.json({
  success: true,
  data: {
    totalUsers,
    paidUsers,
    freeUsers,
    usersByCountry,
    usersByCareerStage,
  },
});
} catch (error) {
  console.error('User analytics error:', error);
  return NextResponse.json(
    { error: 'Failed to fetch user analytics' },
    { status: 500 }
  );
}
}

```

CV Analysis Analytics API

File: src/app/api/analytics/cv-analysis/route.ts

```
import { NextRequest, NextResponse } from 'next/server';
import connectDB from '@lib/db/connection';
import CVAnalysis from '@lib/db/models/CVAnalysis';

export async function GET(request: NextRequest) {
  try {
    await connectDB();

    // Total analyses
    const totalAnalyses = await CVAnalysis.countDocuments();

    // Average CV score
    const avgScoreResult = await CVAnalysis.aggregate([
      {
        $group: {
          _id: null,
          avgScore: { $avg: '$cvScore' },
        },
      },
    ]);
    const averageScore = avgScoreResult[0]?.avgScore || 0;

    // Monthly trend
    const monthlyTrend = await CVAnalysis.aggregate([
      {
        $group: {
          _id: {
            month: { $month: '$analysisDate' },
            year: { $year: '$analysisDate' },
          },
          analyses: { $sum: 1 },
          avgScore: { $avg: '$cvScore' },
        },
      },
      { $sort: { '_id.year': 1, '_id.month': 1 } },
      {
        $project: {
          month: '$_id.month',
          year: '$_id.year',
          analyses: 1,
          avgScore: { $round: ['$avgScore', 1] },
          _id: 0,
        },
      },
    ]);

    return NextResponse.json({
      success: true,
      data: {
        totalAnalyses,
        averageScore: Math.round(averageScore * 10) / 10,
        monthlyTrend,
      },
    });
  } catch (error) {
    return NextResponse.json({ success: false, error: error.message }, { status: 500 });
  }
}
```

```

    },
  });
} catch (error) {
  console.error('CV analysis analytics error:', error);
  return NextResponse.json(
    { error: 'Failed to fetch CV analytics' },
    { status: 500 }
  );
}
}

```

Phase 5: Dashboard UI Components (Day 2-3)

KPI Card Component

File: src/components/dashboard/KPICard.tsx

```

import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { LucideIcon } from 'lucide-react';

interface KPICardProps {
  title: string;
  value: string | number;
  icon: LucideIcon;
  trend?: {
    value: number;
    isPositive: boolean;
  };
  description?: string;
}

export default function KPICard({
  title,
  value,
  icon: Icon,
  trend,
  description
}: KPICardProps) {
  return (
    <Card>
      <CardHeader className="flex flex-row items-center justify-between pb-2">
        <CardTitle className="text-sm font-medium text-muted-foreground">
          {title}
        </CardTitle>
        <Icon className="h-4 w-4 text-muted-foreground" />
      </CardHeader>
      <CardContent>
        <div>{value}</div>
        {trend & & (
          <p>
            {trend.isPositive ? '↑' : '↓'} {Math.abs(trend.value)}% from last month
          </p>
        )}
      </CardContent>
    </Card>
  );
}

```

```

        {description &&& (
          <p>{description}</p>
        )}
      </CardContent>
    </Card>
  );
}

```

Country-wise Chart Component

File: src/components/dashboard/CountryChart.tsx

```

'use client';

import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer } from

interface CountryData {
  country: string;
  count: number;
}

interface CountryChartProps {
  data: CountryData[];
}

export default function CountryChart({ data }: CountryChartProps) {
  return (
    <Card>
      <CardHeader>
        <CardTitle>Users by Country</CardTitle>
      </CardHeader>
      <CardContent>
        <ResponsiveContainer width="100%" height={300}>
          <BarChart data={data}>
            <CartesianGrid strokeDasharray="3 3" />
            <XAxis dataKey="country" angle={-45} textAnchor="end" height={100} />
            <YAxis />
            <Tooltip />
            <Bar dataKey="count" fill="#3b82f6" />
          </BarChart>
        </ResponsiveContainer>
      </CardContent>
    </Card>
  );
}

```

CV Trend Line Chart

File: src/components/dashboard/CVTrendChart.tsx

```
'use client';

import { Card, CardContent, CardHeader, CardTitle } from '@components/ui/card';
import { LineChart, Line, XAxis, YAxis, CartesianGrid, Tooltip, Legend, ResponsiveContainer } from 'recharts';

interface TrendData {
  month: string;
  analyses: number;
  avgScore: number;
}

interface CVTrendChartProps {
  data: TrendData[];
}

export default function CVTrendChart({ data }: CVTrendChartProps) {
  return (
    <Card>
      <CardHeader>
        <CardTitle>CV Analysis Trend</CardTitle>
      </CardHeader>
      <CardContent>
        <ResponsiveContainer width="100%" height={300}>
          <LineChart data={data}>
            <CartesianGrid strokeDasharray="3 3" />
            <XAxis dataKey="month" />
            <YAxis yAxisId="left" />
            <YAxis yAxisId="right" orientation="right" />
            <Tooltip />
            <Legend />
            <Line
              yAxisId="left"
              type="monotone"
              dataKey="analyses"
              stroke="#3b82f6"
              name="Analyses"
            />
            <Line
              yAxisId="right"
              type="monotone"
              dataKey="avgScore"
              stroke="#10b981"
              name="Avg Score"
            />
          </LineChart>
        </ResponsiveContainer>
      </CardContent>
    </Card>
  );
}
```

Phase 6: Deployment (Day 4, Hours 28-32)

Step 1: Prepare for Production

Update `next.config.js`:

```
/** @type {import('next').NextConfig} */
const nextConfig = {
  reactStrictMode: true,
  swcMinify: true,
};

module.exports = nextConfig;
```

Step 2: Push to GitHub

```
git init
git add .
git commit -m "Initial commit: Admin Analytics Dashboard"
git branch -M main
git remote add origin https://github.com/your-username/admin-dashboard.git
git push -u origin main
```

Step 3: Deploy to Vercel

1. Visit vercel.com and sign in
2. Click "New Project"
3. Import your GitHub repository
4. Configure environment variables in Vercel dashboard
5. Deploy

Environment Variables to add in Vercel:

- MONGODB_URI
- JWT_SECRET
- NEXT_PUBLIC_APP_URL

Demo Credentials

Include these in your README:

Admin Account:

- Email: admin@dashboard.com
- Password: Admin@123

Regular User:

- Email: user@dashboard.com
- Password: User@123

Key Features Checklist

- ✓ User authentication with JWT
- ✓ Role-based access control (Admin/User)
- ✓ Protected routes with middleware
- ✓ MongoDB integration with Mongoose
- ✓ Server-side rendering with Next.js
- ✓ Responsive dashboard layout
- ✓ KPI cards for key metrics
- ✓ Interactive charts (Recharts)
- ✓ User demographics visualization
- ✓ CV analysis trends
- ✓ Paid vs Free user segmentation
- ✓ Top users leaderboard
- ✓ Feedback analytics
- ✓ TypeScript for type safety
- ✓ ShadCN UI components
- ✓ Deployment-ready

Performance Optimization Tips

1. **Use SSG for static pages** - Pre-render at build time
2. **Implement data caching** - Cache API responses
3. **Optimize images** - Use Next.js Image component
4. **Code splitting** - Lazy load components
5. **MongoDB indexing** - Add indexes to frequently queried fields

Troubleshooting

MongoDB Connection Issues

- Ensure IP whitelist includes 0.0.0.0/0 in MongoDB Atlas
- Verify connection string format
- Check network restrictions

Authentication Not Working

- Verify JWT_SECRET is set
- Check cookie settings
- Ensure middleware matcher is correct

Charts Not Rendering

- Verify data format matches chart requirements
- Check for client-side rendering with 'use client'
- Ensure ResponsiveContainer is used

Resources

- [Next.js Documentation](#) [1] [2]
- [MongoDB Atlas](#) [3] [4]
- [ShadCN UI](#) [5] [6]
- [Recharts Documentation](#) [7] [8]
- [Vercel Deployment](#) [9] [10]

Good luck with your implementation! ☺

[11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [35] [36] [37] [38] [39] [40]
[41] [42] [43] [44] [45] [46] [47] [48] [49] [50] [51] [52] [53] [54] [55] [56] [57] [58] [59] [60] [61] [62] [63] [64] [65] [66] [67] [68] [69] [70]
[71] [72] [73] [74] [75] [76] [77] [78] [79] [80] [81] [82] [83] [84] [85] [86] [87] [88] [89] [90]

✱✱

1. <https://www.facebook.com/groups/software.dev.ph/posts/10040709902706388/>
2. <https://strapi.io/blog/ssr-in-next-js>
3. <https://www.geeksforgeeks.org/nextjs/how-to-integrate-mongodb-in-next-js/>
4. <https://hevodata.com/learn/next-js-mongodb-connection/>
5. <https://www.traversymedia.com/blog/nextjs-admin-dashboard-project>
6. <https://github.com/satnaing/shadcn-admin>
7. <https://refine.dev/blog/recharts/>
8. <https://github.com/recharts/recharts>
9. <https://www.youtube.com/watch?v=AiiGjB2AxqA>
10. <https://vercel.com/templates/next.js/admin-dashboard>
11. <https://appwrite.io/blog/post/csr-ssg-ssr>
12. <https://www.youtube.com/watch?v=nERpb57brK0>
13. <https://www.youtube.com/watch?v=hhudoSMM0yU>
14. <https://dev.to/yugjadvani/csr-vs-ssr-vs-ssg-vs-isr-a-deep-dive-for-modern-web-development-33kl>
15. https://www.youtube.com/watch?v=IG_mTu0wyZA

16. <https://www.youtube.com/watch?v=S5tjBqzs31w>
17. https://dev.to/hitesh_developer/step-by-step-guide-to-building-an-admin-dashboard-with-nextjs-26e4
18. <https://nextjs.org/docs/pages/building-your-application/rendering/client-side-rendering>
19. <https://github.com/tribixbite/awesome>
20. <https://jasonwatmore.com/next-js-13-app-router-mongodb-user-rego-and-login-tutorial-with-example>
21. <https://npm-compare.com/chart.js,echarts,victory-chart,chartist,recharts,billboard.js,apexcharts>
22. <https://github.com/nextauthjs/next-auth/discussions/9609>
23. <https://github.com/Kiranism/next-shadcn-dashboard-starter>
24. <https://theaverageprogrammer.hashnode.dev/choosing-the-right-charting-library-for-your-nextjs-dashboard>
25. https://www.reddit.com/r/nextjs/comments/1ioifz4/role_based_routing_in_nextjs_middleware_for/
26. <https://www.shadcn.io/template/satnaing-shadcn-admin>
27. <https://www.reddit.com/r/reactjs/comments/1f55kh6/echartshighchartsrecharts/>
28. <https://www.linkedin.com/pulse/role-based-access-control-using-nextjs-middlewares-akasukpe-kkt1f>
29. <https://www.facebook.com/groups/reactjs.developers.community/posts/3926419814295127/>
30. <https://shadcnui.com>
31. <https://strapi.io/blog/chart-libraries>
32. <https://authjs.dev/guides/role-based-access-control>
33. <https://www.creative-tim.com/blog/dashboard/introducing-material-dashboard-shadcn-a-free-modern-admin-dash-board-built-with-shadcn-ui-tailwind-css/>
34. <https://embeddable.com/blog/javascript-charting-libraries>
35. <https://stackoverflow.com/questions/32493271/mongodb-designing-a-statistics-dashboard-schema>
36. <https://www.simplekpi.com/Blog/KPI-Dashboards-a-comprehensive-guide>
37. <https://www.mongodb.com/community/forums/t/how-to-design-schema-for-different-kinds-of-users/224707>
38. <https://www.freelancer.in/job-search/css-image-rendering-cross-browser/3/>
39. <https://datapad.io/blog/dashboard-design>
40. <https://www.sprinkledata.com/blogs/mongodb-vs-mongoose-understanding-the-differences-and-use-cases>
41. <https://adminmart.com/nextjs-best-practices/>
42. <https://www.phdata.io/blog/dashboard-design-essentials-kpi-templates/>
43. <https://dzone.com/articles/real-time-dashboard-with-mongodb>
44. <https://vercel.com/docs/frameworks/full-stack/nextjs>
45. <https://www.justinmind.com/ui-design/dashboard-design-best-practices-ux>
46. <https://www.mongodb.com/community/forums/t/best-schema-for-user-logs-in-analytics-context/11879>
47. <https://dev.to/ethanleetech/best-8-deployment-and-hosting-for-nextjs-dip>
48. <https://nextjs.org/docs/app/api-reference/config/typescript>
49. <https://sourceforge.net/software/product/Launchify/alternatives>
50. <https://dev.to/thatanjan/2-ways-to-set-up-nextjs-with-mongodb-and-mongoose-4afo>
51. <https://www.youtube.com/watch?v=6jQdZcYY8OY>
52. <https://blog.appsignal.com/2023/08/09/how-to-use-mongodb-and-mongoose-for-nodejs.html>
53. <https://www.geeksforgeeks.org/reactjs/create-a-pie-chart-using-recharts-in-reactjs/>

54. <https://blog.stackademic.com/comprehensive-guide-to-next-js-15-app-router-64e967d700f8>
55. <https://stackoverflow.com/questions/70097154/use-schemas-from-mongoose-with-nextjs-and-mongodb>
56. <https://www.youtube.com/watch?v=CO8STl6kNwo>
57. <https://dev.to/moizlokhawala09/building-scalable-web-apps-with-nextjs-15-a-complete-guide-to-the-app-router-and-typescript-3e0j>
58. <https://stackademic.com/blog/a-guide-to-build-an-api-server-with-nextjs-14-and-mongoose-e01f0e10a68a>
59. <https://janhesters.com/blog/how-to-set-up-nextjs-15-for-production-in-2025>
60. <https://www.youtube.com/watch?v=bMHs2pHkbsM>
61. https://developer.salesforce.com/docs/atlas.en-us.bi_dev_guide_json.meta/bi_dev_guide_json/bi_dbjson_wave_designer.htm
62. <https://www.ijfmr.com/papers/2024/6/29890.pdf>
63. <https://stackoverflow.com/questions/77417878/why-environment-variables-inside-env-local-will-show-undefined-when-i-am-using>
64. <https://grafana.com/docs/grafana/latest/dashboards/build-dashboards/view-dashboard-json-model/>
65. <https://github.com/meghnalohani/Resume-Scoring-using-NLP>
66. <https://github.com/mongodb-developer/nextjs-with-mongodb>
67. <https://community.splunk.com/t5/All-Apps-and-Add-ons/Create-dashboards-that-has-json-as-the-dashboard-content-using/m-p/552181>
68. <https://dev.to/hezronokwach/building-a-nextjs-back-end-with-mongodb-for-crud-operations-a-guide-to-testing-with-postman-4ne7>
69. <https://arxiv.org/pdf/2307.15752.pdf>
70. <https://nextjs.org/docs/pages/guides/environment-variables>
71. https://opensource.adobe.com/Spry/samples/data_region/JSONDataSetSample.html
72. <https://blog.loopcv.pro/beat-the-ai-resume-score-algorithm/>
73. <https://github.com/vercel/next.js/discussions/37488>
74. <https://coursedog.freshdesk.com/en/support/solutions/articles/48001237503-requirements-json-data-structure-for-simple-requirements>
75. <https://enhancv.com/resources/resume-checker/>
76. <https://www.offsec.com/blog/cve-2025-29927/>
77. https://www.w3schools.com/howto/howto_css_sidebar_responsive.asp
78. <https://studio3t.com/knowledge-base/articles/build-mongodb-aggregation-queries/>
79. <https://bitskingdom.com/blog/nextjs-authentication-protected-routes/>
80. <https://www.syncfusion.com/react-components/react-sidebar>
81. <https://stackoverflow.com/questions/12702080/mongodb-explain-for-aggregation-framework>
82. <https://projectdiscovery.io/blog/nextjs-middleware-authorization-bypass>
83. <https://dev.to/ramonak/dashboard-layout-with-react-js-and-material-ui-27m4>
84. <https://www.geeksforgeeks.org/mongodb/aggregation-in-mongodb/>
85. <https://embarkingonvoyage.com/blog/next-js-authentication-and-authorization-2025-best-practices-for-secure-web-development/>
86. <https://purecode.ai/community/createdashboardwithmui-tailwind-dashboard>

87. <https://www.mongodb.com/docs/manual/core/aggregation-pipeline/>
88. <https://clerk.com/docs/reference/nextjs/clerk-middleware>
89. <https://www.scribd.com/document/752533028/cektitle>
90. <https://www.material-tailwind.com/docs/react/sidebar>